COMPUTER ENGINEERING
QUEZON CITY

COURSE CODE/TITLE:	CPE 010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 15, 2026	**DATE SUBMITTED:**	October 20, 2026
NAME:	Payas, John Paul S. Reyes, Fenzie Aaron A.	**INSTRUCTOR:**	Engr. Jimlord Quejado

HANDS-ON ACTIVITY NO. 13.1 — **PARALLEL ALGORITHMS AND MULTITHREADING**

1. PROCEDURE OUTPUT

What procedures did you perform?

ILO A – Part 1: Simple One-Threaded Example

Screenshot

ILOA.cpp > ...

```cpp
#include <iostream>
#include <string>
#include <thread>

void print (int n, const std::string &str) {
    std::cout << "------------------------" << std::endl;
    std::cout << "Printing Integer: " << n << std::endl;
    std::cout << "Printing String: " << str << std::endl;
    std::cout << "------------------------" << std::endl;
}

int main() {
    std::thread t1(print, 10, "T.I.P.");
    t1.join();
return 0;
}
```

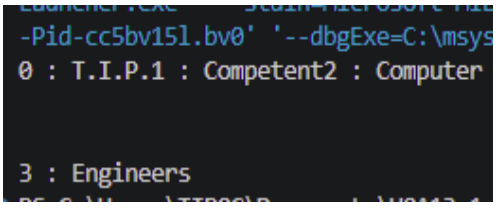
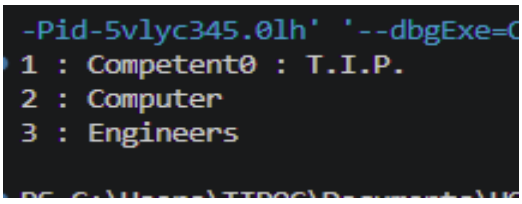
Output

```
-Pid-0pzw4p2l.pqb' '--dbgExe=C:\msys64\ucrt64\bin\gdb.ex
------------------------
Printing Integer: 10
Printing String: T.I.P.
------------------------
PS C:\Users\TIPQC\Documents\HOA13p1_PAYAS_REYES>
```

Analysis

The program creates one thread, t1, to execute the print() function using the values 10 and "T.I.P.". The join() function makes the main thread wait until t1 finishes executing.

Table 13-1. Simple One-Threaded Example

ILO B – Part 2: Multi-Threaded Example	
Screenshot	
Output	 
Analysis	The program creates four threads using a for loop, with each thread receiving a different index and string. The threads execute concurrently, allowing multiple tasks to run independently. The join() function ensures that all threads finish before the program terminates.

```cpp
#include <iostream>
#include <string>
#include <vector>
#include <thread>


void print(int n, const std::string &str) {
    std::string msg = std::to_string(n) + " : " + str;
    std::cout << msg << std::endl;

}

int main() {
    std::vector<std::string> s = {
        "T.I.P.",
        "Competent",
        "Computer",
        "Engineers"
    };

    std::vector<std::thread> threads;

    for (int i = 0; i < s.size(); i++) {
        threads.push_back(std::thread(print, i, s[i]));
    }

    for (auto &th : threads) {
        th.join();
    }

    return 0;
}
```

```
-Pid-cc5bv15l.bv0' '--dbgExe=C:\msys
0 : T.I.P.1 : Competent2 : Computer


3 : Engineers
```

```
-Pid-5vlyc345.0lh' '--dbgExe=C
1 : Competent0 : T.I.P.
2 : Computer
3 : Engineers
```

Table 13-2. Multithreaded Example

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Part A: Demonstrating Parallelism, Concurrency, and Multithreading in C++

PAYAS, JOHN PAUL S.

1. **Write a definition of multithreading and its advantages/disadvantages.**

- Multithreading is a programming technique that allows a program to execute multiple threads concurrently. Its advantages include improving program responsiveness, making better use of CPU resources, and potentially increasing performance on multicore processors. But the disadvantages include the possibility of data races, deadlocks, and unexpected results when threads access shared data without proper synchronization. It can also make programs more difficult to debug and maintain.

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**

- Multithreading is useful in different applications that need to handle multiple tasks efficiently. First, web servers use multithreading to process requests from different users concurrently. Second, file processing programs can use multiple threads to process separate files or portions of data. Third, desktop applications can use one thread to handle user interaction while another performs background tasks, keeping the interface responsive. It can also be applied to data analysis and sorting algorithms to process independent portions of large datasets.

3. **Differentiate between parallelism and concurrency.**

- Concurrency is the ability of a program to manage multiple tasks whose execution periods overlap, even when the tasks take turns using a single CPU core. Parallelism involves executing multiple tasks simultaneously, usually using multiple CPU cores or processing resources. Concurrency focuses on managing multiple tasks, while parallelism focuses on performing multiple operations at the same time.

Part B: Create C++ Code	
Screenshot	

```cpp
#include <iostream>
#include <thread>
#include <mutex>

int globalValue = 0;
std::mutex mtx;

void add(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    globalValue += value;
}

void display() {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << "Global value: " << globalValue << std::endl;
}

int main() {
    std::thread T1(add, 10);
    std::thread T2(add, 20);
    std::thread T3(add, 30);

    display();

    T1.join();
    display();

    T2.join();
    display();

    T3.join();
    display();

    return 0;
}
```

Output

```
Global value: 0
Global value: 60
Global value: 60
Global value: 60
```

Analysis	This demonstrates how multithreading works by creating three threads that add 10, 20, and 30 to a global integer variable. The join() function waits for a specific thread to finish before displaying the value again. Since all three threads start at the same time, they may finish their operations before the first display. This explains why the output can show 60 in every display. The mutex ensures that the shared variable is updated safely.

Part C: Use Multi-threading With One of the Algorithms Previously Developed	
Screenshot	

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <functional>
#include <algorithm>

void merge(std::vector<int>& arr, int left, int mid, int right) {
    std::vector<int> temp;
    int i = left;
    int j = mid + 1;

    while (i <= mid && j <= right) {
        if (arr[i] <= arr[j]) {
            temp.push_back(arr[i++]);
        } else {
            temp.push_back(arr[j++]);
        }
    }

    while (i <= mid) {
        temp.push_back(arr[i++]);
    }

    while (j <= right) {
        temp.push_back(arr[j++]);
    }

    for (int k = 0; k < static_cast<int>(temp.size()); k++) {
        arr[left + k] = temp[k];
    }
}

void parallelMergeSort(std::vector<int>& arr, int left, int right) {
    if (left >= right) {
        return;
    }

    if (right - left < 3) {
        std::sort(arr.begin() + left, arr.begin() + right + 1);
        return;
    }

    int mid = left + (right - left) / 2;
```

```
    std::thread leftThread(
        parallelMergeSort, std::ref(arr), left, mid
    );

    parallelMergeSort(arr, mid + 1, right);

    leftThread.join();

    merge(arr, left, mid, right);
}

int main() {
    std::vector<int> arr = {38, 12, 45, 7, 23, 19, 50, 4};

    std::cout << "Original array: ";
    for (int value : arr) {
        std::cout << value << " ";
    }
    std::cout << std::endl;

    parallelMergeSort(arr, 0, static_cast<int>(arr.size()) - 1);

    std::cout << "Sorted array: ";
    for (int value : arr) {
        std::cout << value << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

```
PS C:\Users\TIPQC\Documents\HOA13> cd
Original array: 29 8 41 15 3 26 12 35
Sorted array: 3 8 12 15 26 29 35 41
```

Analysis	It demonstrates multithreading by dividing an array of eight integers, {29, 8, 41, 15, 3, 26, 12, 35}, into two sections and assigning each section to a separate thread. Thread T1 sorts the first half, while thread T2 sorts the second half using the Bubble Sort algorithm, which compares adjacent elements and swaps them when they are in the wrong order. The join() function ensures that both threads finish their tasks before the program continues. Afterward, the std::sort() function sorts the entire array in ascending order, producing the output 3, 8, 12, 15, 26, 29, 35, 41.

Part A: Demonstrating Parallelism, Concurrency, and Multithreading in C++

REYES, FENZIE AARON A.

1. **Write a definition of multithreading and its advantages/disadvantages.**

- Multithreading is a programming technique that allows a computer or program to run multiple threads with each thread handling different tasks. Through this technique, it can enhance the use of CPU resources, handling multiple tasks efficiently, and potentially faster execution. However, we need to be also aware of multithreading disadvantages before using the technique. This includes more complex code, possible data races when threads access the same data, and extra memory and processing cost. (Reference: HOA 13 – Parallel Algorithms and Multithreading)

---

2. **Rationalize the use of multithreading by providing at least 3 use-cases.**

- **Multithreading** is useful when a program needs to handle multiple tasks efficiently as stated in the previous answer. Three examples of this technique are:
- **Web Servers:** Handle requests from multiple users at the same time.
- **File Processing:** Process multiple files or separate parts of a large task using different threads.
- **User Interfaces**: Keep an application responsive while another task runs in the background. Through these, we can put the multithreading in good use if properly executed.

---

3. **Differentiate between parallelism and concurrency.**

- Concurrency means multiple tasks make progress during the same period, even if the CPU switches between them. On the other hand, parallelism means multiple tasks are actually executed at the same time, usually using multiple CPU cores. In other words, concurrency focuses on managing multiple tasks, while parallelism focuses on performing tasks simultaneously. Concurrency allows a program to handle several tasks efficiently by switching between them, especially when some tasks are waiting for input or other resources. Meanwhile, parallelism divides tasks so that different processors or CPU cores can execute them at the same time, potentially reducing the total execution time. For example, a program that downloads multiple files by switching between download tasks demonstrates concurrency, while a program that processes different parts of a large dataset using multiple CPU cores demonstrates parallelism. Although both concepts improve program efficiency, concurrency is mainly concerned with coordinating multiple tasks, whereas parallelism is concerned with increasing performance by executing multiple tasks simultaneously.

REFERENCES:

GeeksforGeeks. (2026, May 4). *Difference between concurrency and parallelism*. https://www.geeksforgeeks.org/operating-systems/difference-between-concurrency-and-parallelism/

Richard, R. M. (n.d.). Module 13: Parallel Algorithms and Multithreading. CPE010 Data Structures and Algorithms, Technological Institute of the Philippines – Quezon City.

Part B: Create C++ Code	
Screenshot	

```cpp
#include <iostream>
#include <thread>
#include <mutex>

int globalValue = 0;
std::mutex mtx;

void add(int value) {
    std::lock_guard<std::mutex> lock(mtx);
    globalValue += value;
}

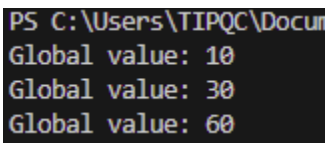
void display() {
    std::lock_guard<std::mutex> lock(mtx);
    std::cout << "Global value: " << globalValue << std::endl;
}

int main() {
    std::thread T1(add, 10);
    T1.join();
    display();

    std::thread T2(add, 20);
    T2.join();
    display();

    std::thread T3(add, 30);
    T3.join();
    display();

    return 0;
}
```

Output	
Analysis	It creates three threads that add 10, 20, and 30 to a global integer variable. Each thread is joined before the next value is displayed, resulting in predictable totals of 10, 30, and 60. The mutex protects the shared variable during updates and prevents multiple threads from modifying it simultaneously. The join() function ensures that each thread finishes its task before the program proceeds to the next step.

Part C: Use Multi-threading With One of the Algorithms Previously Developed	
Screenshot	

```cpp
#include <iostream>
#include <vector>
#include <thread>
#include <algorithm>
#include <functional>

void bubbleSort(std::vector<int>& arr, int start, int end) {
    for (int i = start; i < end; i++) {
        for (int j = start; j < end - (i - start); j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap(arr[j], arr[j + 1]);
            }
        }
    }
}

int main() {
    std::vector<int> arr = {29, 8, 41, 15, 3, 26, 12, 35};

    std::cout << "Original array: ";
    for (int x : arr) {
        std::cout << x << " ";
    }
    std::cout << std::endl;

    int mid = arr.size() / 2;

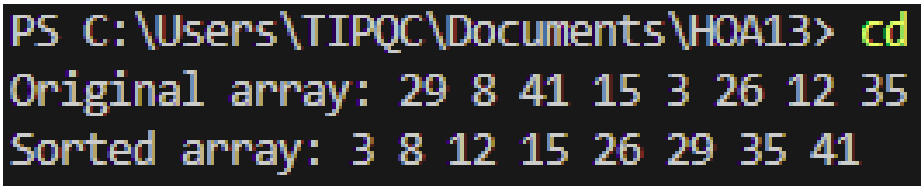
    std::thread T1(bubbleSort, std::ref(arr), 0, mid - 1);
    std::thread T2(bubbleSort, std::ref(arr), mid, arr.size() - 1);

    T1.join();
    T2.join();

    std::sort(arr.begin(), arr.end());

    std::cout << "Sorted array: ";
    for (int x : arr) {
        std::cout << x << " ";
    }
    std::cout << std::endl;

    return 0;
}
```

	
Analysis	It divides the array into two sections and uses separate threads to sort each section with Bubble Sort. After both threads finish using join(), std::sort() arranges the entire array in ascending order. The output demonstrates that the numbers are sorted correctly. The final sorting step is performed by std::sort(), so this is a demonstration of multithreading rather than a fully parallel Bubble Sort algorithm.

CONCLUSION (PAYAS)

In this activity, we learned about parallelism, concurrency, and multithreading. We gained a better understanding of how threads execute tasks, how the join() function manages thread execution, and how mutexes protect shared variables from race conditions. Through the supplementary activities, we applied multithreading to a global integer variable and a sorting algorithm. We also learned that multithreading can improve program performance when tasks can be processed independently, although it may introduce additional overhead and synchronization challenges.

CONCLUSION (REYES)

Through this activity, I learned how multithreading works by creating separate threads to perform tasks. In Part B, I learned how to use a mutex at the header file in order to protect a shared variable, and how join() function which ensures that each thread finishes before the program continues. In Part C, I learned how to divide an array into two sections and use separate threads to sort each section using Bubble Sort. The program then uses std::sort() to arrange the entire array in ascending order. Overall, this activity helped me understand the importance of thread synchronization, concurrency, and multithreading in improving program efficiency.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: ChatGPT

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: Grammar correction and assistance in understanding the instructions and concepts related to the activity.

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: It was only used for checking grammar errors and providing assistance when needed. The coding, analysis, and completion of the activity were reviewed and finalized by us

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.